

LockSnake – Programmierpraktikum 2

Lambda-Ausdrücke, Methodenreferenzen und Observer-Pattern

1. Projektziel

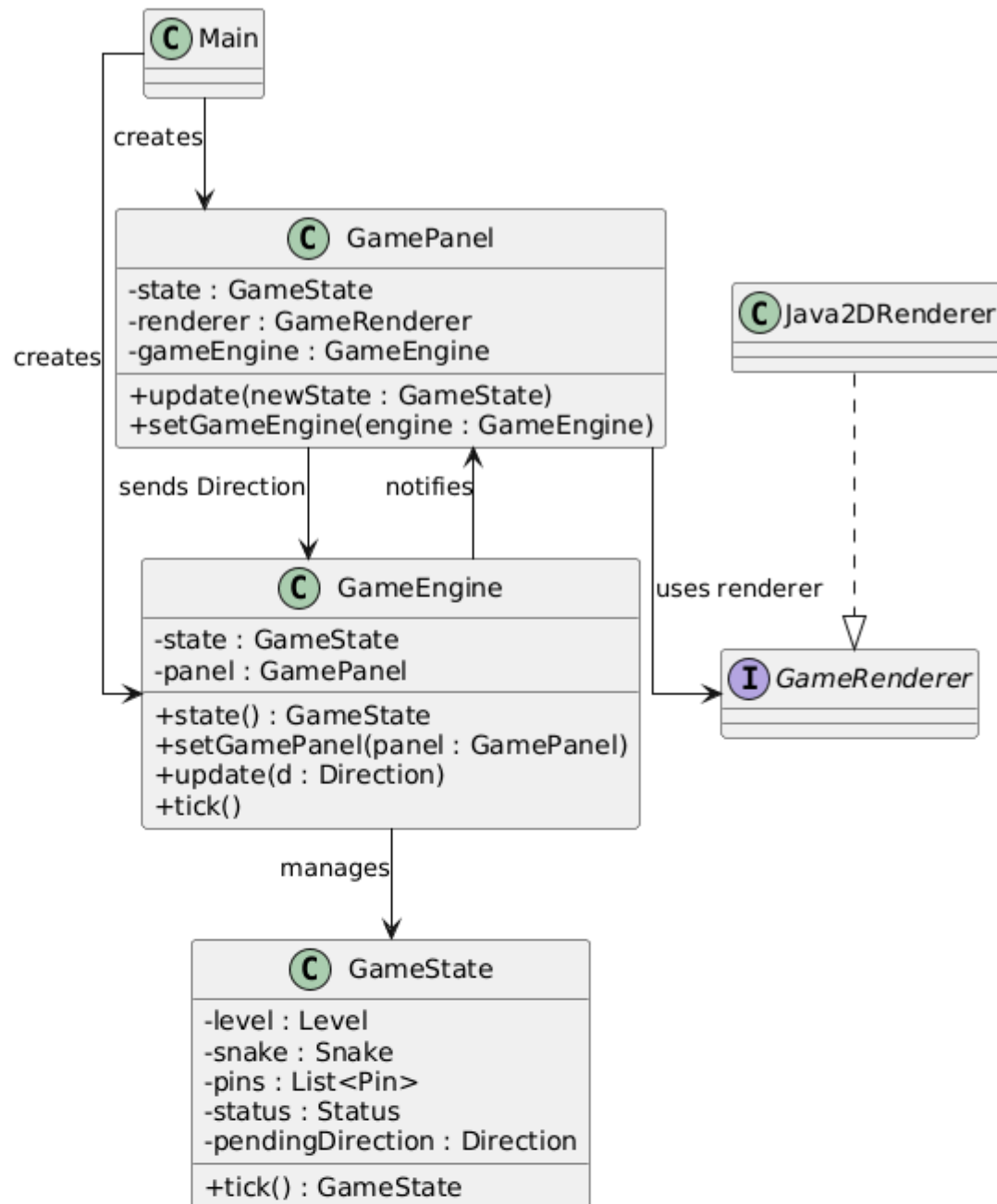
Ziel des Projekts war die Implementierung eines kleinen Swing-basierten Spiels nach dem Vorbild von „Mouse P.I. Lockpicking“, bei dem eine Snake durch ein Spielfeld navigiert, Pins aktiviert und dabei Kollisionen, Wände sowie falsche Aktivierungsrichtungen vermeiden muss.

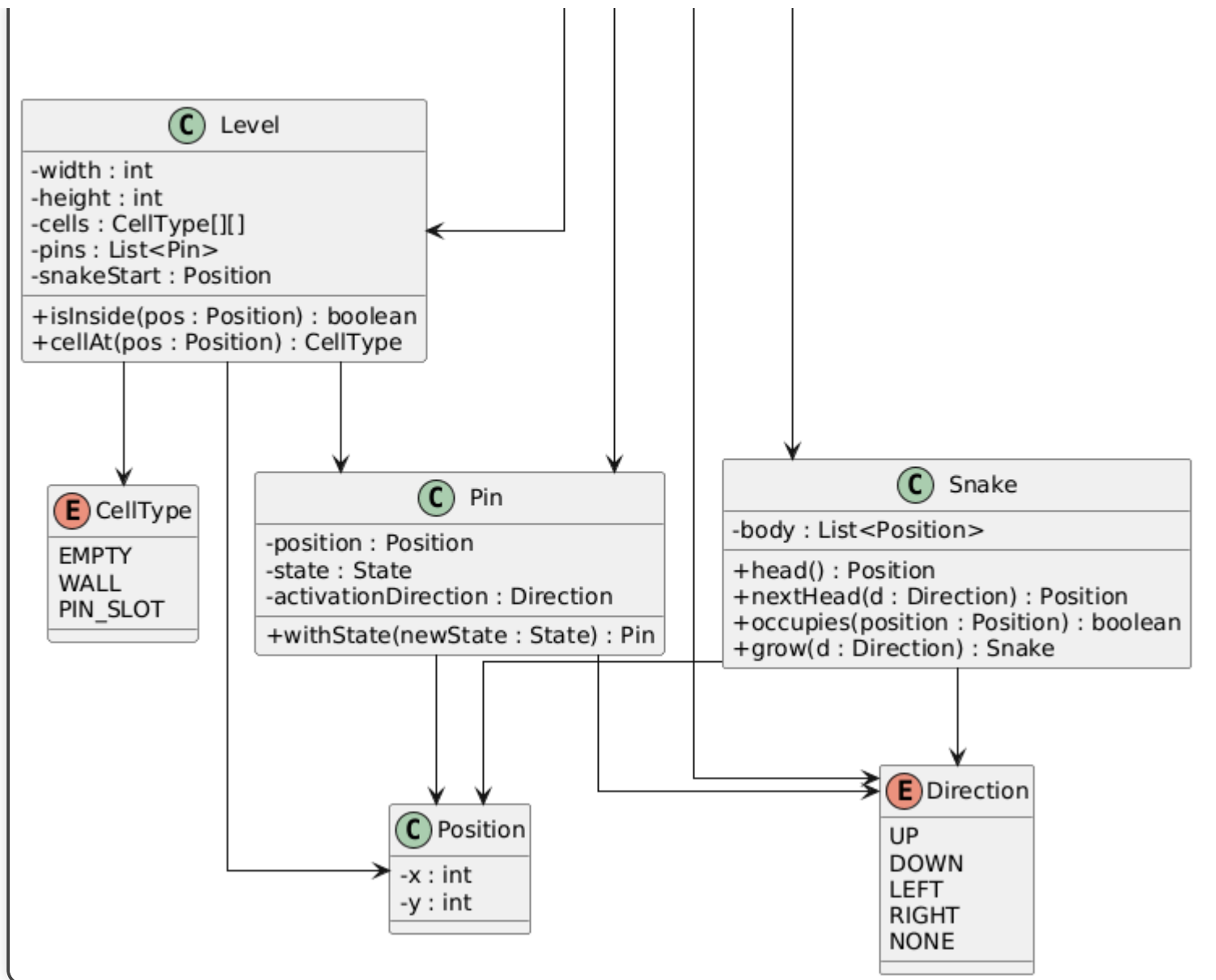
Im Fokus standen dabei die praktische Anwendung von Lambda-Ausdrücken, Methodenreferenzen, Observer-Pattern, objektorientierter Modellierung sowie automatisierten JUnit-Tests.

2. UML-Klassendiagramm

Das UML-Klassendiagramm zeigt die Beziehungen zwischen den zentralen Klassen und Komponenten des Projekts:

LockSnake - UML-Klassendiagramm





Hinweis: Das Diagramm visualisiert insbesondere die Interaktion zwischen Main, GameEngine, GamePanel, GameState sowie den Modellklassen Snake, Pin, Level, Position und Direction.

3. Implementierung von GameState

Die zentrale Spiellogik wurde in der Methode `tick()` umgesetzt. Diese verarbeitet pro Tick genau einen möglichen Spielschritt und prüft dabei sämtliche Spielregeln.

- ✓ Keine Bewegung ohne gesetzte Blickrichtung
- ✓ Bewegung auf freiem Feld
- ✓ Wände blockieren Bewegung
- ✓ Spielfeldrand führt zu Game Over
- ✓ Selbstkollision führt zu Game Over
- ✓ LOW-Pins können nur aus korrekter Richtung aktiviert werden
- ✓ HIGH-Pins blockieren wie Wände
- ✓ Gewinnbedingung: alle Pins aktiviert

4. Implementierung von GameEngine

Die GameEngine verwaltet den aktuellen GameState, verarbeitet Richtungsänderungen, führt Tick-Aktualisierungen aus und stellt die Verbindung zwischen Eingabe und GUI her.

- ✓ Initialisierung des Spielzustands
- ✓ Verarbeitung von Tastatureingaben
- ✓ Tick-basierte Spielaktualisierung

- ✓ **Observer-Benachrichtigung an GamePanel**

5. Observer-Pattern

UI → Engine (Direction-Eingabe)

```
gameEngine.update(direction);
```

Engine → UI (State-Update)

```
panel.update(state);
```

Dadurch wird die Benutzeroberfläche automatisch neu gezeichnet, sobald sich der Spielzustand verändert.

6. Lambda-Ausdrücke und Methodenreferenzen

Verwendete Lambda-Ausdrücke:

- `() -> { ... }` für `SwingUtilities.invokeLater`
- `e -> { ... }` für Timer-Loop

- keyCode -> ... für KeyBindings

Verwendete Methodenreferenzen:

- this::setupKeyBindings
- Pin::state
- Pin.State::isSet

7. JUnit-Tests

Zur Validierung der Spiellogik wurden 10 unterschiedliche Testfälle für GameState implementiert:

- ✓ Keine Bewegung bei NONE
- ✓ Normale Bewegung
- ✓ Wand blockiert
- ✓ Out of Bounds
- ✓ Selbstkollision
- ✓ Kein Tick nach Spielende
- ✓ LOW-Pin korrekt
- ✓ LOW-Pin falsche Richtung
- ✓ HIGH-Pin blockiert
- ✓ Gewinnbedingung

Alle 10 Tests wurden erfolgreich bestanden.

8. Fazit

Das Projekt erfüllt sämtliche Kernanforderungen des Praktikums: vollständige Spielmechanik, Observer-Pattern, moderne Java-Konstrukte (Lambda-Ausdrücke und Methodenreferenzen), UML-Dokumentation sowie automatisierte Tests.

Besonders relevant war dabei das Zusammenspiel zwischen Architektur, Wartbarkeit und funktionaler Korrektheit. Durch die Kombination aus Modellierung, GUI-Anbindung und Testautomatisierung wurde ein vollständiges, nachvollziehbares und erweiterbares Softwaresystem umgesetzt.